Deep Learning on the Go

Mir Afgan Talpur[1], **Faisal Z. Qureshi**[1]

[1]Ontario Tech University – Visual Computing Lab – Faculty of Science
2000 Simcoe St N. – Oshawa – Ontario – Canada

Mir.Talpur@ontariotechu.net, Faisal.Qureshi@ontariotechu.ca

Abstract. *Smartphones have become a large part of our daily lives. Many of us rely on them for our camera needs as their photographic capabilities have advanced substantially over the past decade. In that time, deep learning has taken strides to revolutionize computational photography. For these reasons, we are interested in utilizing deep learning networks on smartphones to enhance smartphone photography. This thesis focuses on porting neural style transfer networks to mobile. Utilizing TensorFlow Lite to port these networks, we were able to create Pastiche, a style transfer Android application. Pastiche combines style transfer and masking techniques to create different styles of art. We conclude that although deep learning networks are currently non-trivial to run on smartphones, advances in optimization techniques and smartphone hardware suggest a bright future for deep learning on smartphones.*

Keywords*: Style Transfer, TensorFlow Lite, Android Development, On-Device Inference.*

I would like to thank my supervisor, Dr. Faisal Qureshi, for his guidance throughout my undergraduate honours thesis. I would also like to thank the Visual Computing Lab and the Faculty of Science at Ontario Tech University for providing me with the resources to complete this project.

Contents

1 Introduction **4**

2 Related Works **5**

2.1 Neural Style Transfer 5

2.2 Deep Learning on Mobile 6

2.3 Quantization 6

3 Resources and Tools **8**

3.1 TensorFlow Lite 8

3.1.1 Choose Your Model 8

3.1.2 Convert Your Model 8

3.1.3 Run Inference With Your Model 11

3.1.4 Optimize Your Model 11

3.2 ML Kit 11

3.3 Android Studio 12

4 Model **13**

5 Methodology **15**

5.1 Image Inpainting 15

5.2 Neural Style Transfer 15

5.3 Application Development 15

5.4 Sample Pastiches 20

5.4.1 Normal Mask 20

5.4.2 Inverted Mask 22

5.4.3 Regular Neural Style Transfer 23

6 Conclusion **25**

7 Future Work **26**

7.1 Short-Term Future Work 26

7.2 Long-Term Future Work 26

8 Listing Citations **27**

9 Figure Citations **28**

1 Introduction

As smartphone use has seen substantial growth over the past decade across the global market, smartphone photography has become part of our daily lives. Photo and video applications have seen the second largest growth amongst mobile application downloads over the past year and have seen the most growth in time spent in-app between 2016 – 2018 with 210% growth [1]. This can likely be attributed to improvements in smartphone camera technology, ranging from hardware such as improved sensors, to more versatile software and applications that can utilize smartphone cameras more effectively. Consumers around the world have begun to rely on their smartphones for photography, picture editing, and sharing their photos across various social media platforms.

In this time, deep learning has also seen tremendous growth, particularly in the field of computational photography. The use of Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Generative Adversarial Networks (GANs), have revolutionized how we approach computer vision. These networks have allowed for major breakthroughs in areas such as object classification, pose detection, image inpainting, and style transfer.

As both fields rapidly grow and evolve, the next step in deep learning and mobile development is porting deep neural networks to smartphones. However, these deep neural network models are often computationally expensive and resource intensive. Moreover, smartphones have traditionally been viewed as constrained systems, with limitations in processing power and memory capacity. Despite these hindrances, recent advances have been made in efficiently compressing deep neural network models, and frameworks such as TensorFlow Lite have been specifically created with the goal of on-device inference [2].

With this in mind, this thesis will explore how utilizing deep learning can enhance the smartphone photography experience. In particular, we will use TensorFlow Lite to port neural style transfer networks to our Android application, Pastiche.

2 Related Works

This section will introduce previous studies and works that relate to porting neural style transfer to mobile. Over the past few years, neural style transfer has become a very popular topic in the field of computer vision. During this time, many studies have also investigated the use of deep learning networks on mobile platforms. These studies provide many insights with regards to how neural style transfer can be performed in a constrained system. They also show that the introduction of TensorFlow Lite and quantization have made deep learning on mobile much more feasible.

2.1 Neural Style Transfer

Style transfer refers to the task of blending a content image with a style image in order to create a new image called a pastiche. First introduced by Gatys *et al.*, neural style transfer involves performing this task using deep neural networks [3]. This process involved obtaining the style representation of the style image and the content representation of the content image, both of which would be combined to produce a pastiche. Obtaining the style representation of the image was done by utilizing a feature space that was originally meant for capturing texture information [3]. Using filter responses of each layer in the network, they were able to create a multi-scale representation of the style image, capturing its texture information but not its arrangement [3]. The content representation, on the other hand, was obtained through the feature responses in higher layers of the network, which capture high-level content with relation to objects and their arrangement [3].

The major contribution of the paper was that these representations in the CNN were separable [3]. In spite of this, the researchers also found that the content and style of an image could not be perfectly disentangled. To overcome this limitation, they were able to emphasize the reconstruction of either an images content or its style by utilizing a loss function that contains terms for image content and image style, which are well separated [3]. This way, for a given pair of images, they were able to emphasize one image's content and emphasize the other's style when combining them to create a stylized image.

However, the Gatys *et al.* algorithm was very computationally expensive [4]. Others tried to improve on this by focusing on less expensive implementations through introducing feedforward style transfer networks [5, 6]. Unfortunately, this technique loses the flexibility of the Gatys *et al.* paper as these networks are tied to a single style [4]. Eventually, Ghiasi *et al.* worked on a new method for performing fast neural style transfer by training a style transfer model on a very large dataset, allowing this model to generalize to previously unobserved paintings [7]. This paper combined the speed of fast style transfer networks with the flexibility of the Gatys *et al.* paper in order to create a real-time stylization technique using any pair of content and style images [7].

The style prediction network in the Ghiasi *et al.* paper produced a low dimensional representation for artistic styles, which was mostly organized by visual and semantic similarity, truly capturing the artistic range of a given artist [7]. With this improved architecture, this model was able to perform neural style transfer on any content/style image pair in real time [7]. For these reasons, this architecture acts as the basis for the networks used in our mobile application, as discussed further in section 4.

2.2 Deep Learning on Mobile

Deep learning has revolutionized computational photography and has paved the path for innovation in computer vision. Porting this technology to mobile is of much interest to smartphone manufacturers and mobile application developers. Numerous works have investigated this idea through various deep learning tasks, including image classification [8], object detection [9], and bokeh simulation [10]. In earlier studies, implementing state-of-the-art models on mobile was challenging as these networks were not optimized for on-device inference [11].

This began changing with the introduction of frameworks like TensorFlow Lite [11]. This is a deep learning framework designed for on-device inference on constrained systems such as mobile devices and embedded systems. TensorFlow Lite supports a variety of deep learning operations from TensorFlow and allows them to be ported to a smartphone. This is done through efficient optimization and storage in a compressed flat buffer format [2]. However, TensorFlow Lite is a relatively new framework resulting in a number of operations remaining unsupported [12].

In terms of network architecture, researchers at Google created an efficient class of models called MobileNets for the purpose of mobile vision applications [8]. These networks and their successors have been applied to a variety of tasks ranging from object detection to semantic segmentation [13]. These CNNs used depthwise separable convolutions, which resulted in much greater efficiency in network architecture [8]. Because of their use of small, highly efficient models to run complicated tasks, MobileNets ended up being very popular for many deep learning projects on mobile.

The two main hindrances for deep learning on mobile are the limitations of mobile hardware and the computational burden associated with a deep learning network. Fortunately, smartphone hardware manufacturers and deep learning engineers are working towards solutions to these obstacles. Recently, smartphone chipset manufacturers including Qualcomm, Samsung, and MediaTek have begun to include hardware acceleration through neural processing units (NPUs) and AI accelerators [11]. These accelerators have resulted with substantially improved inference speeds, rivalling that of entry-level CUDA-enabled desktop GPUs [11]. These improvements are very promising for the future of deep learning on smartphones, suggesting that the limitations today might not be an issue in the near future.

With regards to the computational burden of deep learning networks, as the demand for these complicated networks has increased, more efficient implementations of these networks have become necessary. In order to solve these efficiently compressed and optimized networks, researchers have turned to optimization techniques like quantization.

2.3 Quantization

As previously mentioned, neural networks can be very resource intensive and computationally expensive. This is a major roadblock to deep learning on mobile as smartphones are constrained in terms of computational resources. One solution to dealing with large networks is optimization through quantization. Quantization is an optimization technique that reduces the size of networks, often at the cost of accuracy, by reducing the precision of the values and calculations in the network [12]. This can include converting float32

values to float16 or int8, resulting in quicker execution times, reduced model sizes, and reduced RAM consumption [11].

Quantization has become a focus in optimizing networks for smartphones as it can drastically improve the trade-off between accuracy and on-device latency [14]. Both of these aspects are crucial for on-device inference. A low accuracy defeats the purpose of using deep learning networks for these complicated tasks and poor latency can lead to a disappointing user experience. Improvements in quantization have made tasks such as classification feasible on mobile by maintaining accuracies on integer-only quantized MobileNets as compared to floating-point MobileNets [14].

However, quantization is far from perfect. Although new techniques have improved on the accuracy trade-off associated with the quantization process, this is still a major concern for tasks that require high accuracies. Additionally, several quantization techniques require completely retraining networks from scratch [11]. Lastly, mobile GPUs are designed for floating point operations. These GPUs consume less power and generate less heat than the CPU for equivalent tasks [15]. Integer quantized models run better on CPU, which suggests that they cannot take advantage of the more efficient GPU hardware, resulting in a poorly optimized experience. Running the full float32 model on GPU is seen as an alternative to quantization when accuracy loss is unavoidable in the quantization process. However, the increased file size and RAM consumption remain in this case.

3 Resources and Tools

We will now discuss the resources and tools that were required to create Pastiche, our neural style transfer application for Android. The application was created on Android Studio using TensorFlow Lite models which were deployed through ML Kit. These tools are discussed in detail below.

3.1 TensorFlow Lite

Tensorflow Lite is a deep learning framework for on-device inference [2]. It allows users to convert a TensorFlow model, optimize it, and deploy it on constrained devices such as mobile devices and embedded systems. In order to use a TensorFlow Lite model, the following steps are taken:

3.1.1 Choose Your Model

There are three options for choosing a model:

1. Re-training a model
2. Training a custom model
3. Using a pre-trained model

Re-training a model involves a process referred to as transfer learning [12]. This process allows you to use a trained model and re-train it for performing another task. Transfer learning is more efficient than training a model from scratch as it requires less time and data than the former process [12].

Certain tasks require custom TensorFlow models. If this is the case, they must first be converted into the TensorFlow Lite format, *.tflite*. This process is described in next subsection.

Lastly, you can use a pre-trained TensorFlow Lite model. This is the option we chose for this thesis. TensorFlow Lite offers a selection of pre-trained models that are capable of solving a variety of deep learning tasks. In the case of Pastiche, we opted to use the pre-trained neural style transfer networks [16]. These networks were optimized and ready for deployment onto mobile systems.

3.1.2 Convert Your Model

For the first two options mentioned earlier, you must first convert the models to *.tflite* before deploying them to a mobile device. As previously discussed, deep learning networks can be quite resource intensive, which can hinder their effectiveness on mobile and embedded systems. TensorFlow Lite was designed to help execute these large network models efficiently on these constrained systems.

The TensorFlow Lite converter is the tool that converts TensorFlow Models to the TensorFlow Lite format. The converter uses a special format to store models, which reduces

file size and adds additional optimizations that do not impact the model's accuracy [12]. Additionally, more options are available that allow for further efficiency at the cost of accuracy and precision. These will be discussed in subsection 3.1.4. However, it should be noted that TensorFlow Lite does not currently support certain operations from TensorFlow.

We will focus on two methods for converting TensorFlow models to TensorFlow Lite using `tf.lite.TFLiteConverter`, a Python API for TensorFlow Lite conversion [17]. The following examples are found in the TensorFlow Lite Converter documentation [17].

The first option for converting a TensorFlow model to TensorFlow Lite involves converting a SavedModel directory. For this choice, you would use the `TFLiteConverter.from_saved_model()` method, as seen in listing 1 [17]. Before converting in this case, you must first save your TensorFlow model using `tf.saved_model.save`. After you have saved your TensorFlow model, you can then use the `TFLiteConverter.from_saved_model()` method to convert your model.

Listing 1. Converting SavedModel Directory to TensorFlow Lite

```
import tensorflow as tf

# Construct a basic model.
root = tf.train.Checkpoint()
root.v1 = tf.Variable(3.)
root.v2 = tf.Variable(2.)
root.f = tf.function(lambda x: root.v1 * root.v2 * x)

# Save the model.
export_dir = "/tmp/test_saved_model"
input_data = tf.constant(1., shape=[1, 1])
to_save = root.f.get_concrete_function(input_data)
tf.saved_model.save(root, export_dir, to_save)

# Convert the model.
converter = tf.lite.TFLiteConverter.from_saved_model(export_dir)
tflite_model = converter.convert()
```

You can also choose to convert a Keras model to TensorFlow Lite. Here, you would use the `TFLiteConverter.from_keras_model()` method, as shown in listing 2 [17]. With this option, you simply use your Keras model as the argument for the `TFLiteConverter.from_keras_model()` method to convert your model.

Listing 2. Converting Keras Model to TensorFlow Lite

```
import tensorflow as tf

# Create a simple Keras model.
x = [-1, 0, 1, 2, 3, 4]
y = [-3, -1, 1, 3, 5, 7]

model = tf.keras.models.Sequential(
    [tf.keras.layers.Dense(units=1, input_shape=[1])])
model.compile(optimizer='sgd', loss='mean_squared_error')
model.fit(x, y, epochs=50)

# Convert the model.
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()
```

Building on the Keras conversion example above, listing 3 shows how to convert a pre-trained MobileNet model in Keras to TensorFlow Lite [17]. This example also runs inference on the model and compares the results with the original model. Here, a pre-trained MobileNetV2 model is used as the argument to the `TFLiteConverter.from_keras_model()` method. Following the conversion, the TensorFlow Lite model is then loaded to an interpreter and tensors are allocated. Once the input and output details are retrieved, the model is tested on random input data. After inference has completed, the original model is also tested on random input data. Lastly, the results from both models are compared.

Listing 3. Converting and Comparing Keras MobileNet Model to TensorFlow Lite

```
import numpy as np
import tensorflow as tf

# Load the MobileNet tf.keras model.
model = tf.keras.applications.MobileNetV2(
    weights="imagenet", input_shape=(224, 224, 3))

# Convert the model.
converter = tf.lite.TFLiteConverter.from_keras_model(model)
tflite_model = converter.convert()

# Load TFLite model and allocate tensors.
interpreter = tf.lite.Interpreter(model_content=tflite_model)
interpreter.allocate_tensors()

# Get input and output tensors.
input_details = interpreter.get_input_details()
output_details = interpreter.get_output_details()

# Test the TensorFlow Lite model on random input data.
input_shape = input_details[0]['shape']
input_data = np.array(np.random.random_sample(input_shape), dtype
   =np.float32)
```

```
interpreter.set_tensor(input_details[0]['index'], input_data)

interpreter.invoke()

# The function `get_tensor()` returns a copy of the tensor data.
# Use `tensor()` in order to get a pointer to the tensor.
tflite_results = interpreter.get_tensor(output_details[0]['index'
    ])

# Test the TensorFlow model on random input data.
tf_results = model(tf.constant(input_data))

# Compare the result.
for tf_result, tflite_result in zip(tf_results, tflite_results):
  np.testing.assert_almost_equal(tf_result, tflite_result,
      decimal=5)
```

3.1.3 Run Inference With Your Model

Using your model, the TensorFlow Lite Interpreter (available on both Android and iOS platforms), and input data, you are able to run inference on your mobile or embedded system. After entering the input data, the interpreter will run inference on the model, then provide access to the output. Some smartphones may provide hardware acceleration for these operations, allowing for substantial speed gains.

3.1.4 Optimize Your Model

TensorFlow Lite allows you to perform optimizations on your model, resulting in smaller model sizes and performance increases, with minimal impact on accuracy. This can be done through quantization. This process reduces precision of values and operations in a model by converting full floating point values to half-precision floats or integers. However, this process may result in a loss of accuracy, depending on your model. This loss is minimal in many models.

As previously mentioned, deep learning models can be large and resource exhaustive. Depending on the task at hand, TensorFlow Lite can be a very useful framework for deploying these TensorFlow models on constrained systems, such as a smartphone. Again, the effectiveness of this process is dependent on the model and the operations it contains.

3.2 ML Kit

ML Kit, currently in beta release, is an SDK from Firebase that uses Google's machine learning technologies to allow deep learning network models on Android and iOS platforms [18]. These machine learning technologies include Google Cloud Vision API, TensorFlow Lite, and the Android Neural Networks API. ML Kit offers production-ready APIs for numerous deep learning tasks including, but not limited to, face detection, text detection, and barcode scanning [18].

Depending on the model, ML Kit can run on-device when there is no network connection available or use cloud-based processing for increased accuracy when it can access a network. Unfortunately, this is not currently supported on custom models. Another feature of ML Kit is that it can be used to deploy custom models. ML Kit was created for Firebase, allowing custom models to be uploaded to Firebase. Firebase can host and serve the model to your app, with ML Kit acting as an API layer that can run the model in your application. This can allow for smaller application install sizes, over-the-air model updates, and dynamic model selection for specific user segments [19]. These features are particularly useful in Pastiche, as they allow for scalability, upgradability, and efficiency.

3.3 Android Studio

Android Studio is Android's official integrated development environment (IDE), built specifically for developing Android applications. The standout feature of Android Studio is its emulator. Using the emulator, you can prototype and test your application, while manipulating a variety of hardware features. The emulator allows you to push incremental changes without restarting your application while testing. Additional benefits of Android Studio include organizing the structure of the application, testing on various screen sizes, and acting as an all-in-one tool, from developing to deploying the application. Built for efficient development, the IDE has a further variety of tools to aid you during the application build cycle. These include Firebase integration and the Layout Editor, which were very beneficial in developing the functionality and design for Pastiche.

4 Model

Pastiche allows the user to choose any content image and style image which will be blended together to produce a pastiche. This is done through the help of two neural networks, a style prediction network and a style transfer network. These networks were based on the network architecture from the Ghiasi *et al.* paper [7]. Figure 1 shows the architecture for the model originally used by Ghiasi *et al.* The style prediction network, P, creates a style bottleneck vector, $\vec{S}$, from the inputted style image, s. This bottleneck vector and the content image c, are used as inputs for the style transfer network, T, which produces the stylized image, $T(c, \vec{S})$. These two networks were available as integer weight-quantized pre-trained models from TensorFlow Lite [16].

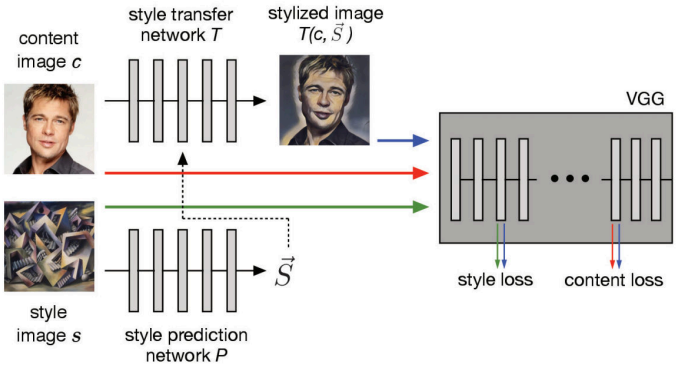


Figure 1. Architecture of the style transfer model, as seen in [7].

The model used by Ghiasi *et al.*, shown above, featured a style transfer network that was based off of the feedforward style transfer network designed by Dumoulin *et al.* [4]. This study found that a style transfer network can be created as an encoder/decoder architecture by using conditional instance normalization [4, 7]. Conditional instance normalization, a procedure created by Dumoulin *et al.*, refers to the process of transforming a layer's activations into a normalized activation specific to a particular painting style [4]. By specializing normalization parameters specific to each style, the style transfer network can model multiple styles at the same time [4, 7]. This allows for a fast and fairly flexible approach to style transfer. However, this technique was limited to only work on the styles explicitly trained on [7]. Ghiasi *et al.* extended this idea to work with many more styles, including stylization for previously unobserved painting styles through a style prediction network [7].

Ghiasi *et al.* utilized a pre-trained Inception-v3 architecture for the style prediction network [7]. They calculated the mean across each activation channel of the Mixed_6e layer of the Inception-v3 model, which has a stride size of 16 pixels [20], resulting in a feature vector with a dimension of 768 [7]. Following this, they compressed the representation using two fully connected layers, the first of which contained 100 units [7]. This resulted in the final style bottleneck vector, $\vec{S}$.

The main advantage of this architecture is that the style prediction network generates a style bottleneck vector. This allows the model to generalize to an unobserved style

image through this style embedding [7]. Ghiasi *et al.* found that jointly training both networks on a large dataset, containing approximately 80,000 paintings and photographs, was sufficient for this model [7]. They found that training on a small amount of paintings results in poor generalization, as opposed to training on a large number of paintings and pictures, which results in the ability to generalize to previously unseen styles [7].

The model that we utilized for Pastiche was created by training the model on a large dataset with data augmentation [21]. The importance of a large dataset, as described above, is that it allows for better generalization. This architecture utilizes a pre-trained checkpoint for the MobileNetV2 model. This model was pre-trained on the Painter by Number dataset (PBN) [22] and the Describable Textures Dataset (DTD) [23] for the style images dataset and trained on ImageNet [24] for the content image dataset. As discussed in section 2.1, Ghiasi *et al.* demonstrated the importance of training at a large scale for improving the ability of the model to generalize to unobserved paintings [7]. This is a crucial aspect of Pastiche, as we want the user to have the freedom to choose any style of painting for the style transfer process.

As discussed further in section 5, poor network latency can result in an undesirable user experience. When creating Pastiche, we prioritized quick runtimes, a desirable accuracy, and a visually appealing user interface. The performance of the style prediction and style transfer networks decide how well we can achieve the first two priorities. Performance benchmarks ran by the TensorFlow Lite team show that these networks operate fairly quickly [16]. The style prediction network completes in approximately 6.7 ms and the style transform network completes in 405 ms [16]. These tests were completed on a Google Pixel 4 smartphone [16], which has an identical system on a chip (SoC) as our testing device, the Samsung Galaxy S10 Plus [25, 26]. Both phones use the Qualcomm Snapdragon 855 SoC, which includes a 64-bit octa-core processor [25, 26, 27]. These times are reflected in the application as the style prediction network runs very quickly in the background after the style is chosen. However, the style transfer network usually runs for a few seconds after the user initiates inference. A float16 quantized version of each network may increase these speeds and provide higher accuracies on smartphones with capable GPUs.

5 Methodology

This section will walk through the development and decision-making behind Pastiche, our neural style transfer application for Android. The overarching goal of this thesis was to explore how deep learning can enhance smartphone computational photography. This would be accomplished through developing an application that utilizes deep learning to perform computer vision related tasks. We attempted this using two different computer vision tasks: image inpainting and neural style transfer.

5.1 Image Inpainting

Our first attempt at porting a deep learning network model to mobile involved a generative image inpainting network. Generative image inpainting refers to the process of using deep learning to recreate and restore missing parts of an image. This network used a generative adversarial network, or GAN, to accomplish this task. GANs use two networks, a generator and a discriminator. The two compete as the former creates new data, while the latter verifies the data in a minimax game [28]. This back and forth between the networks can lead to very realistic data generated by the GAN.

As discussed in section 3.1.2, there are currently a number of unsupported operations in TensorFlow Lite. Unfortunately, this was the case with the generative image inpainting model we were interested in. It appears that networks with this level of architectural complexity are not currently supported as TensorFlow Lite is still a relatively new framework. However, there is currently work being done to develop lightweight GANs and TensorFlow Lite is working on including more operations in future builds, implying there may be a possibility for image inpainting on mobile in the near future [12, 29]. Nevertheless, given the goal of this thesis, we turned our attention from image inpainting to another popular area of deep learning within computer vision, neural style transfer.

5.2 Neural Style Transfer

Of the various popular tasks within computational photography, neural style transfer caught our eyes. Discussed in further detail in section 2.1, neural style transfer, or neural artistic style transfer, involves blending the content of one image with the style of another, resulting in a pastiche. A pastiche, which our application is named after, refers to a non-parodical artistic work that imitates another work. As mentioned in section 3.1.1, TensorFlow Lite has several pretrained models, including a few networks for neural style transfer. We ended up choosing the integer quantized models as they had the smallest file size. However, this reduced file size came at the cost of accuracy due to the limitations of quantization. These two networks, the style prediction network and the style transfer network, were both pre-trained, converted, and pre-optimized, meaning they were both ready for on-device inference.

5.3 Application Development

We opted to develop Pastiche on Android first as it has the largest market share in mobile operating systems worldwide by a large margin, currently sitting at approximately 72% [30]. The benefit of the resources and tools we used to create Pastiche is that many of

them will be compatible with iOS as well, so recreating the application for Apple devices remains a possibility for future work. Knowing the required shapes for the inputs to both the style prediction network and the style transfer network, we first tested whether or not both networks would work on the phone.

The first network, the style prediction network, required an input style image of size (1, 256, 256, 3) and outputted a style bottleneck vector of size (1, 1, 1, 100). The second network, the style transfer network, required an input content image of size (1, x, x, 3), where x is a natural number, as well as the style bottleneck vector from earlier. This network would output the final stylized image, or pastiche, of size (1, x, x, 3).

We used the ML Kit interpreter to run the networks in the application. Currently, we are not taking advantage of the hosting and serving feature, but ML Kit was chosen at the interpreter at this stage to allow remote hosting and serving through Firebase in the future. Sample content and style images were cropped to a 1:1 aspect ratio, with the style images being resized to the required 256 x 256 size. At this stage of testing, the content images were resized to 500 x 500.

For each network, the ML Kit interpreter was set up and its input and output options were defined. Each model was then loaded. The sample input images were represented as bitmaps and were converted to the correctly sized input arrays mentioned earlier. Beginning with the style prediction network, the style image input was then added to the interpreter, which then ran inference using the style prediction network model. The output of the style prediction network was the style bottleneck vector, which would later be used as one of the inputs to the style transfer network. This process was then repeated for the style transfer network. The content image and style bottleneck vector were added as inputs to the interpreter, which then ran inference using the style transfer network model. The output from this network was then converted to a bitmap and rendered to the screen. This entire process was repeated for several different content image sizes, but the emulator would return a memory error when using the style transfer network with any content image size greater than (1, 600, 600, 3). This size was chosen to standard size for all content images in the final app design to maximize image resolution without sacrificing stability.

Upon successfully testing the inputs and outputs for both networks, we began development on the user interface of the application. The testing described above was primarily done through manually adjusting variables and cropping sample images externally, which was a very lengthy process. One of the main advantages of mobile applications is that these apps are designed to make tasks quick and simple in an intuitive, flexible design. With this in mind, our aim for the user interface and functionality of Pastiche was to make it as quick, clean, and simple as possible. This would include automating the process of adjusting variables and allowing users to perform image cropping in-app, so users would not be limited to pre-cropped images. Pastiche would also need to allow users to choose from existing pictures in their gallery and taking a picture through the camera in-app, allowing more flexibility and choice when using the application.

With the intent of developing a well-functioning application with an intuitive user interface, we started creating the main aspect of Pastiche; functioning style transfer. In order to allow flexibility with regards to what images can be chosen, we had to let the users

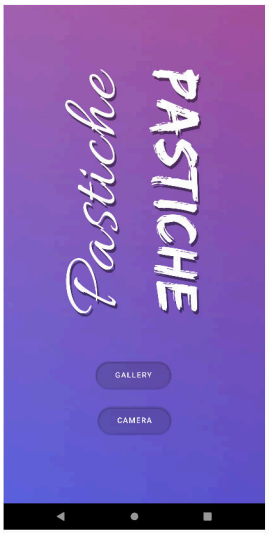


Figure 2. Pastiche's home screen allows the user to select between an existing image in their gallery or taking a photo with the camera.

choose between camera and gallery, as well as incorporate an image cropping operation within the application. The home screen, shown in figure 2, allows the user to either take a new picture with their camera or choose a picture from their gallery. After they have selected or taken their picture, they move on to the cropping screen. We decided to use the Android Image Cropper API for the cropping screen as it allowed the user to easily choose which section of their image they want included and allowed us to set a mandatory 1:1 aspect ratio [31]. With this aspect ratio, once the user has decided on which area of their picture to include as the content image, we simply resized the image to the 600 x 600 size that we set for all content images, as seen in figure 3. For the style image, once the user chooses the area of the picture to represent the style image, it is resized to 256 x 256 as it also uses the same 1:1 aspect ratio as the content image.

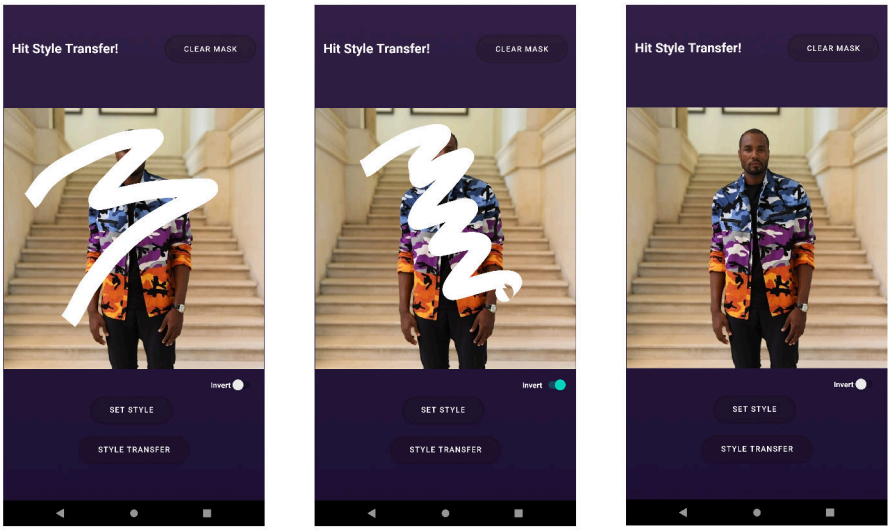


Figure 3. Content images. Left: Normal Mask, Centre: Inverted Mask, Right: No Mask, Direct Style Transfer.

As mentioned earlier, the style prediction network first requires an input style image, which then outputs a style bottleneck vector. The style transfer network uses this bottleneck vector, along with the content image, as an input for the style transfer network. However, Pastiche lets the user choose their content image first, followed by the style image. Again, this was a choice made with ease of use and simplicity in mind. This design choice allows the user to have an idea of what their content image looks like right before applying their style image. This also allows the user to return and choose another style if they were not satisfied with their previous choice.

Once the user has chosen their style image, Pastiche has all the requirements for functioning neural style transfer. As soon as the user approves the crop on their style image by clicking the crop button shown in figure 4, the style image is automatically resized and fed into the style prediction network. This process is nearly instantaneous and happens in the background of the application, meaning the user does not need to worry about initiating this step. After this step is complete and the style bottleneck vector is created, the option for style transfer becomes available.

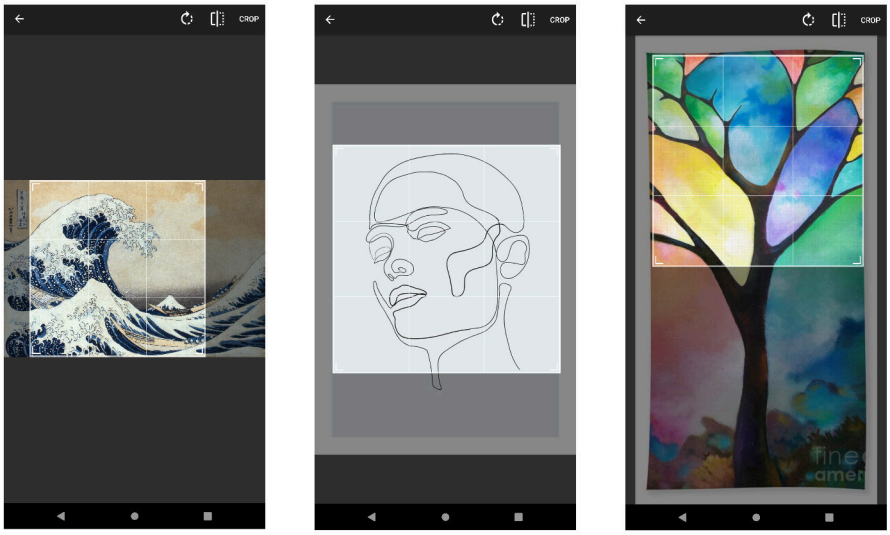


Figure 4. Style images being cropped. Left: Normal Mask, Centre: Inverted Mask, Right: No Mask, Direct Style Transfer.

While both networks could be run consecutively, we opted to let the user choose when to begin inference on the second network. There were a few reasons for this decision. Firstly, we wanted to allow the user to select another style if they had a change of heart after selecting their initial choice. We also wanted to avoid extended loading times while the networks ran in the background. Giving this break after the style prediction network completes contributes to a better user experience as it allows the user to control the pace of the application. Lastly, we were aiming to add a masking feature later on, which this break allowed.

At this stage of development, the neural style transfer worked flawlessly. The user was easily able to choose their own content and style images with abundant flexibility of size and picture origin. They were able to crop the images and feed them into the networks, resulting in their finished pastiches. Again, all the fine details such as resizing

and controlling the inputs and outputs for each network were done behind the scenes. Upon completion of the neural style transfer aspect of the application, the last feature we were planned on adding was a masking feature.

The masking feature would allow users to have even more flexibility and creativity with their art creation. Masking is a compositing technique that allows you to blend multiple images together by hiding pixels. To incorporate this into Pastiche, we used the path, canvas, and paint tools from Android's graphics library to allow for a painting experience. This is a technique used for some drawing and painting applications. We limited the drawing canvas to the content image so the user can preview where their mask will appear in the final image. Once the user draws out a path, we can use this as a mask for their pastiche.

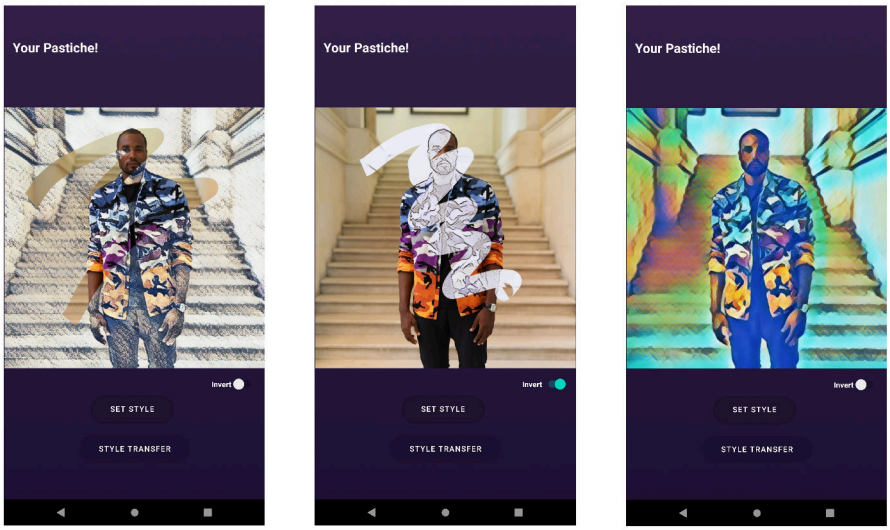


Figure 5. Final pastiches after style transfer. Left: Normal Mask, Centre: Inverted Mask, Right: No Mask, Direct Style Transfer.

At this stage, we had reached a minor dilemma as we had to decide which image we would mask between the original content image and the final pastiche. Again, we wanted the user to have a simple, yet flexible experience with Pastiche, so we decided to give the user the option to choose. This way, they could choose whether to mask the pastiche or the content image based on their preference. Figure 3 shows how they could mask the content image by simply drawing on the canvas or could choose to mask the pastiche image by activating the inverse switch. The results of the style transfer on these images and masks can be seen in figure 5. This functionality added an additional element of entertainment to Pastiche as it allowed the user to have a much more personalized experience in creating their pastiches. This was a much more interactive design than solely relying on the deep learning networks to make their creation. Of course, the option to perform simple style transfer remains as the user can choose not to draw a mask at all.

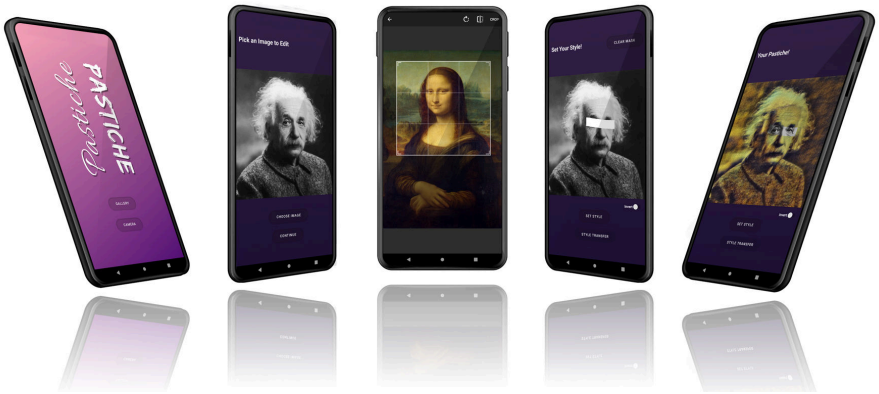


Figure 6. Final design of Pastiche.

Figure 6 shows the final design of Pastiche. Pastiche manages to incorporate an intuitive user interface while utilizing two quantized neural networks for neural style transfer. This application shows that small, efficient networks can be ported to mobile with relative ease. However, it should be noted that these networks used operations compatible with TensorFlow Lite and do not run well on mobile GPUs as a result of integer quantization.

5.4 Sample Pastiches

In the figures below, you will find examples of pastiches generated through our application. Each subsection focuses on different variations of masking techniques, including normal masking, inverted masking, and no masking (regular neural style transfer). These examples demonstrate the variety of pastiches that can be created using neural style transfer and different masking techniques.

5.4.1 Normal Mask

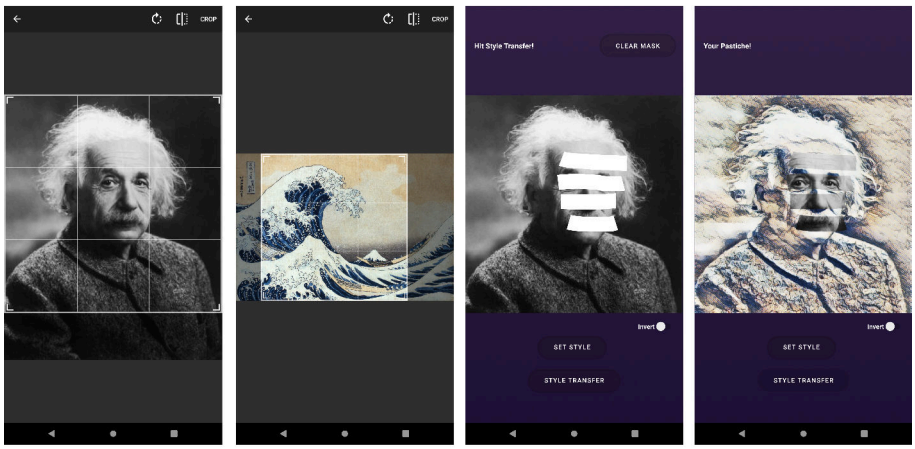


Figure 7. Normal Mask, example 1.

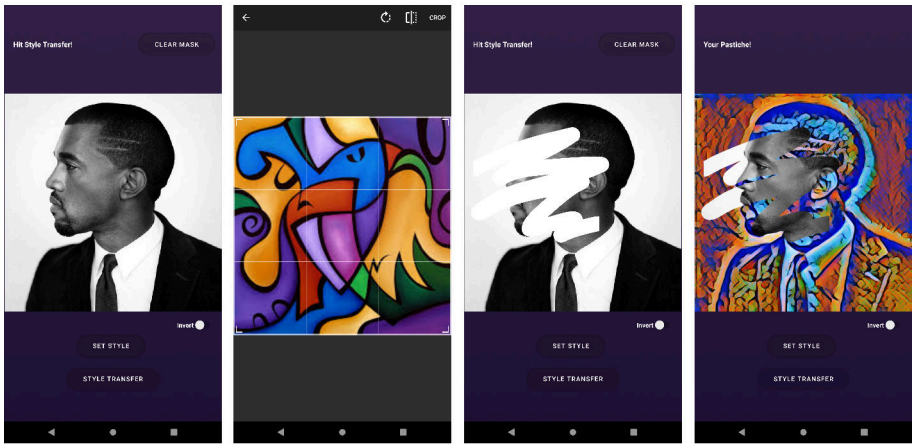


Figure 8. Normal Mask, example 2.

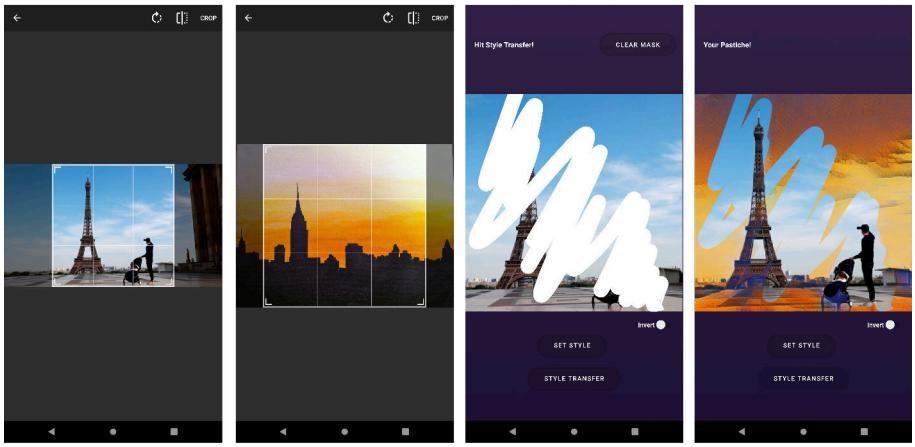


Figure 9. Normal Mask, example 3.

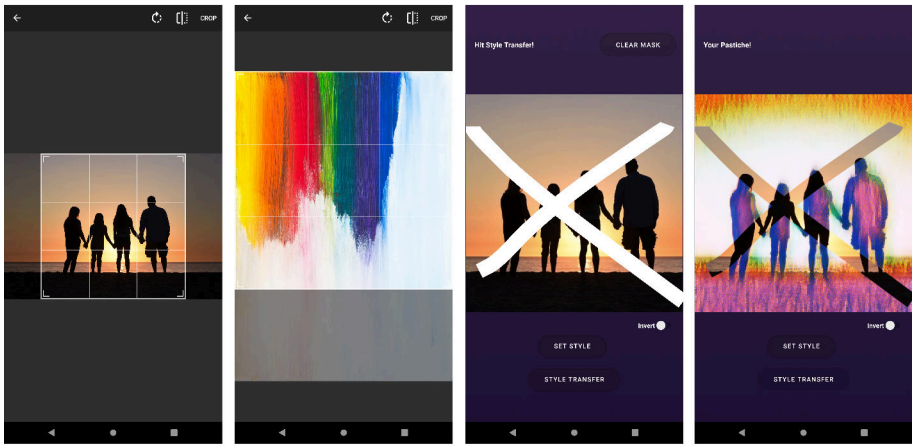


Figure 10. Normal Mask, example 4.

5.4.2 Inverted Mask

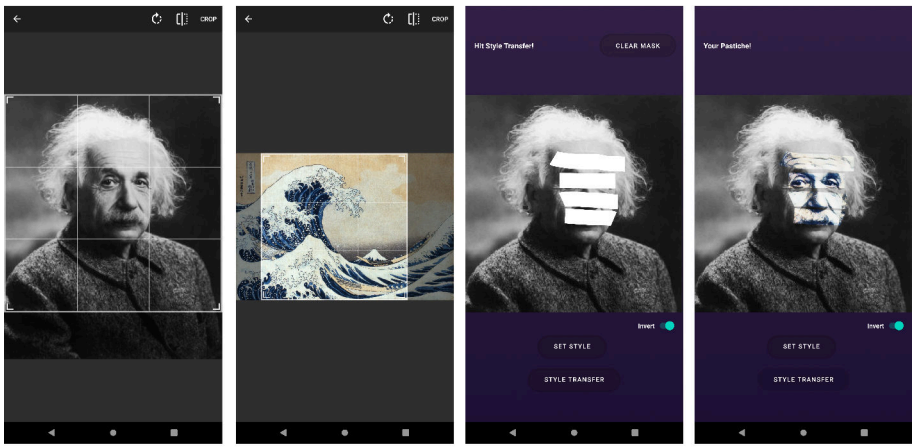


Figure 11. Inverted Mask, example 1.

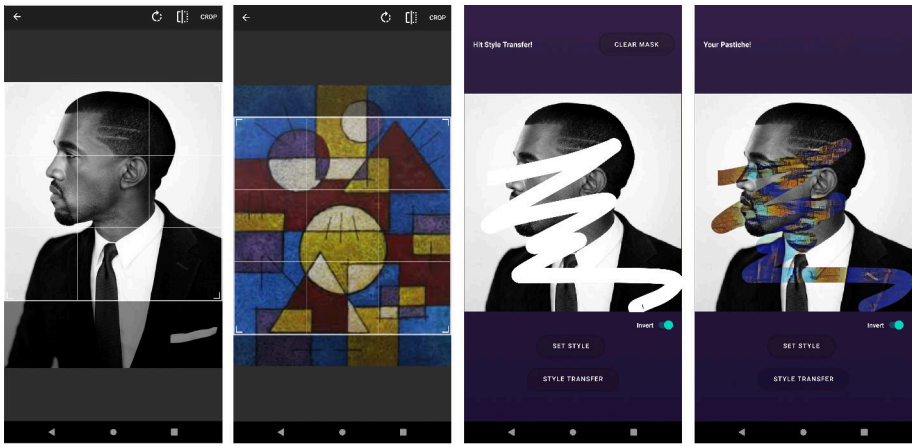


Figure 12. Inverted Mask, example 2.

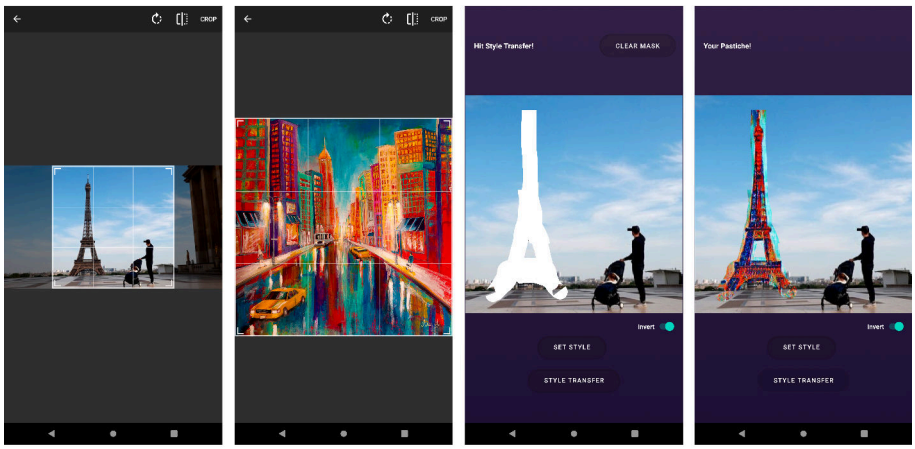


Figure 13. Inverted Mask, example 3.

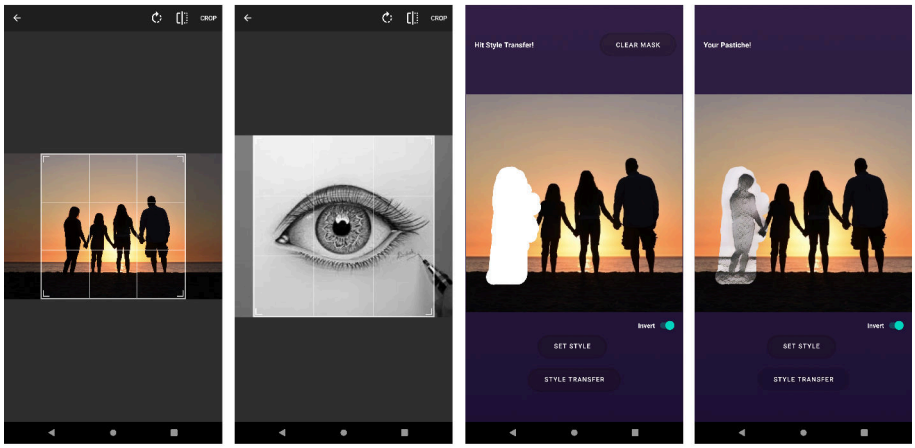


Figure 14. Inverted Mask, example 4.

5.4.3 Regular Neural Style Transfer

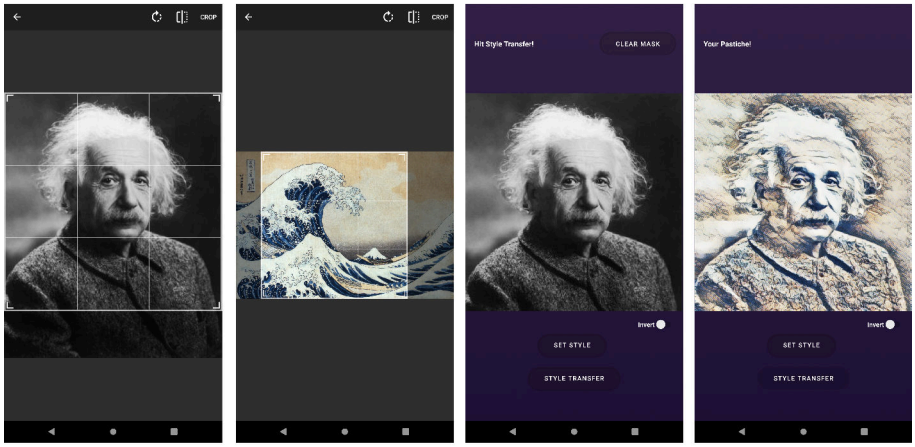


Figure 15. Regular neural style transfer (no mask), example 1.

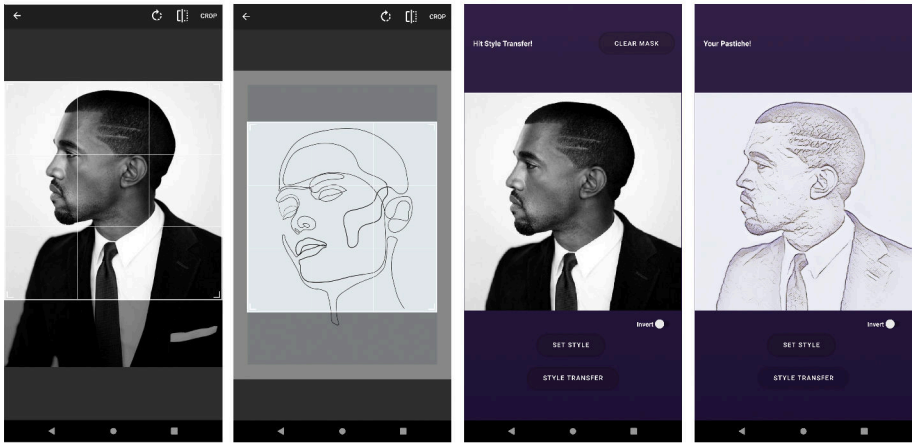


Figure 16. Regular neural style transfer (no mask), example 2.

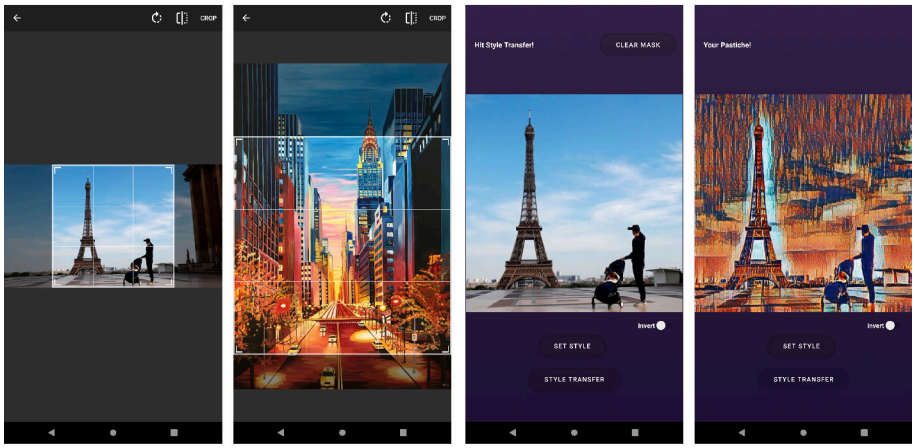


Figure 17. Regular neural style transfer (no mask), example 3.

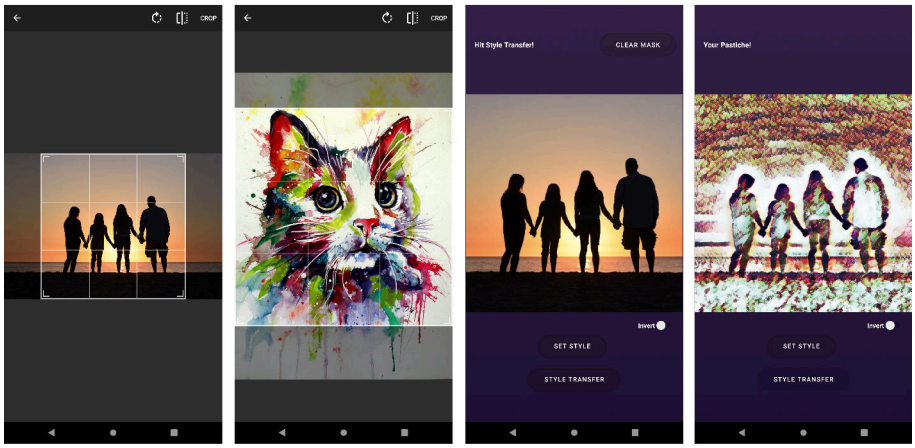


Figure 18. Regular neural style transfer (no mask), example 4.

6 Conclusion

Deep learning networks are often computationally expensive and smartphones remain relatively constrained systems. For these reasons, deep learning networks are non-trivial to run on mobile devices. Our experience with the generative image inpainting network shows this might especially apply to GAN models. With the computational and memory limitations of smartphones, on-device inference is heavily affected.

Luckily, the introduction of TensorFlow Lite is making this difficult task much simpler. Through the addition of optimization techniques such as quantization and simple conversion from the TensorFlow format to the TensorFlow Lite format, deep learning on mobile is becoming more readily available. For certain tasks, including neural style transfer, integer quantized pre-trained networks are hosted through TensorFlow Lite. This feature makes the process of loading a deep learning network much smoother by allowing users to easily add these networks to their application, as we were able to do with Pastiche. However, there are some operations that currently remain unsupported in TensorFlow Lite. Until all TensorFlow operations are supported, porting certain models will continue to be difficult.

Although Pastiche runs very smoothly with the use of quantized networks, some memory limitations remain with regards to image size. Certain tasks require much larger, more complicated architectures. Unfortunately, the fate of some of these tasks on mobile will likely rely on further optimizations and support from frameworks like TensorFlow Lite. As mentioned earlier, certain operations remain unsupported on TensorFlow Lite for the time being and some network architectures may be too architecturally complex to be supported on mobile. Until these networks are compressed and optimized more efficiently, deep learning networks will remain non-trivial to run on smartphones.

Nevertheless, advances in both deep learning and smartphone design show the potential of deep learning on the go in the future. TensorFlow Lite plans to expand their supported operations in future releases. Furthermore, researchers continue to improve on the accuracy trade-off with optimization techniques. Lastly, smartphone manufactures are including more powerful AI accelerators with each successive smartphone design. With improvements in optimization techniques like quantization and the inclusion of stronger, more efficient AI accelerators on mobile phones every year, the future of deep learning on mobile looks very promising.

7 Future Work

7.1 Short-Term Future Work

In the short-term future, Pastiche can be improved in few ways. This could include building upon the current application by incorporating additional types of masks, such as regular polygons. In addition to offering a wider variety of mask shapes, customizing mask features can add further flexibility to the app. These features can include customizations such as alpha blending, feathering edges, and hue, saturation, and luminance (HSL) settings.

An idea that has intrigued us upon completion of Pastiche was incorporating a post-composite user interface in a future build. This would involve pre-computing a variety of different pastiches using various styles for individual content images. In the final stage of the application, the user can then use the mask brush currently integrated in Pastiche to act as a paint brush for each style. The user would be able to select a style and use the brush to paint over the content image, revealing the pastiche in that style in real-time. Again, this feature would further improve the flexibility of Pastiche, providing an even more engaging experience to the user.

Due to time constraints, Pastiche was only created for Android. As discussed in section 5.3, many of the tools and resources used to create Pastiche are also compatible with iOS. Using ML Kit and the same TensorFlow Lite models, this application can be recreated in iOS. Android was the focus for this project as it has the largest market share, but iOS has the second largest. Porting the application to iOS is crucial to reach a larger audience. Moreover, it can allow us to test if both mobile platforms and hardware affect how well a deep learning network runs on mobile.

7.2 Long-Term Future Work

In terms of long-term future work, there remains unexplored aspects of neural style transfer on mobile. Particularly, integrating style transfer to videos on smartphones. While there are likely several hurdles to overcome before accomplishing this task, two aspects of computational photography appear to be the largest hindrances: solving temporal consistency and spatial coherence. Overcoming these issues will be crucial to successfully applying neural style transfer to videos.

While we have shown how neural style transfer can successfully run on smartphones, there are several tasks in computational photography that can benefit from transferring to mobile platforms. Tasks such as image inpainting, photorealistic image generation, and semantic image-to-photo translation can become much more accessible to a larger audience if they become available on mobile platforms. The interest in photo and video applications is rapidly growing and the integration of deep learning networks to enhance computational photography techniques in these applications can revolutionize this segment.

8 Listing Citations

The following listings were borrowed from the TensorFlow Lite *Converter Python API Guide* documentation:

Listing 1 - Converting SavedModel Directory to TensorFlow Lite:
Python Example Downloaded From: *https://www.tensorflow.org/lite/convert/python_api*

Listing 2 - Converting Keras Model to TensorFlow Lite:
Python Example Downloaded From: *https://www.tensorflow.org/lite/convert/python_api*

Listing 3 - Converting Keras MobileNet Model to TensorFlow Lite:
Python Example Downloaded From: *https://www.tensorflow.org/lite/convert/python_api*

9 Figure Citations

Figure 1 - Network Architecture Figure:
Image Downloaded From: *https://arxiv.org/abs/1705.06830*

Figure 3 - Serge Ibaka Content Image:
Image Downloaded From: *https://www.zimbio.com/photos/Serge+Ibaka/Valentino+Front+Row+Paris+Fashion+Week+Menswear/IzIdzDVyk9k*

Figure 4, 7, 11, 15 - The Great Wave off Kanagawa Style Image:
Image Downloaded From: *https://www.cnn.com/style/article/hokusai-great-wave-ukiyo-e-woodblock/index.html*

Figure 4, 16 - Line Art Style Image:
Image Downloaded From: *https://www.postery.com/us/face-line-art-poster*

Figure 4 - Tree of Light Abstract Style Image:
Image Downloaded From: *https://fineartamerica.com/featured/original-art-abstract-art-acrylic-painting-tree-of-light-by-sally-trace-fine-art-sally-trace.html?product=beach-sheet*

Figure 6, 7, 11, 15 - Einstein Content Image:
Image Downloaded From: *https://www.space.com/15524-albert-einstein.html*

Figure 8, 12, 16 - Kanye West Content Image:
Image Downloaded From: *https://i.pinimg.com/originals/d0/31/cb/d031cb61b709247930ea44c6425f9532.jpg*

Figure 8 - Energized (Abstract Art) Style Image:
Image Downloaded From: *https://fineartamerica.com/featured/energized–abstract-art-by-fidostudio-tom-fedro–fidostudio.html*

Figure 9, 13, 17 - Eiffel Tower Content Image:
Image Downloaded From: *https://www.usatoday.com/story/travel/advice/2020/04/03/life-after-coronavirus-when-we-know-its-safe-travel-again/5105038002/*

Figure 9 - City Skyline Style Image:
Image Downloaded From: *https://i.pinimg.com/originals/f2/76/23/f2762315719b6176ddbb1e601b60c3b1.jpg*

Figure 10, 14, 18 - Family Sunset Content Image:
Image Downloaded From: *https://psiloveyou.xyz/how-to-make-peace-with-your-family-73b0fe632a4*

Figure 10 - Rainbow Paint Style Image:
Image Downloaded From: *https://unsplash.com/photos/k0rVudBoB4c*

Figure 12 - Geometric Abstract Art Style Image:
Image Downloaded From: *https://www.novica.com/p/signed-geometric-abstract-painting-from/297775/*

Figure 13 - The City Style Image:
Image Downloaded From: *https://fineartamerica.com/featured/the-city-gray-artus.html*

Figure 14 - Pencil Eye Style Image:
Image Downloaded From: *https://www.facebook.com/deepaksart/*

Figure 17 - Chrysler Building Style Image:
Image Downloaded From: *https://www.1stdibs.com/art/paintings/landscape-paintings/angela-wakefield-large-painting-chrysler-building-new-york-city-nyc-leading-british-artist/id-a_3690702/*

Figure 18 - Colourful Cat Style Image:
Image Downloaded From: *https://i.pinimg.com/originals/0a/72/b7/0a72b7c17448ef822f446529f8df53db.jpg*

References

[1] State of Mobile 2019 - App Annie.

[2] TensorFlow Lite: ML for Mobile and Edge Devices.

[3] Leon A. Gatys, Alexander S. Ecker, and Matthias Bethge. A neural algorithm of artistic style. *CoRR*, abs/1508.06576, 2015.

[4] Vincent Dumoulin, Jonathon Shlens, and Manjunath Kudlur. A learned representation for artistic style. *CoRR*, abs/1610.07629, 2016.

[5] Dmitry Ulyanov, Vadim Lebedev, Andrea Vedaldi, and Victor S. Lempitsky. Texture networks: Feed-forward synthesis of textures and stylized images. *CoRR*, abs/1603.03417, 2016.

[6] Justin Johnson, Alexandre Alahi, and Fei-Fei Li. Perceptual losses for real-time style transfer and super-resolution. *CoRR*, abs/1603.08155, 2016.

[7] Golnaz Ghiasi, Honglak Lee, Manjunath Kudlur, Vincent Dumoulin, and Jonathon Shlens. Exploring the structure of a real-time, arbitrary neural artistic stylization network. *CoRR*, abs/1705.06830, 2017.

[8] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. Mobilenets: Efficient convolutional neural networks for mobile vision applications. *CoRR*, abs/1704.04861, 2017.

[9] Kittipat Apicharttrisorn, Xukan Ran, Jiasi Chen, Srikanth V. Krishnamurthy, and Amit K. Roy-Chowdhury. Frugal following: Power thrifty object detection and tracking for mobile augmented reality. In *Proceedings of the 17th Conference on Embedded Networked Sensor Systems*, SenSys '19, page 96–109, New York, NY, USA, 2019. Association for Computing Machinery.

[10] Neal Wadhwa, Rahul Garg, David E. Jacobs, Bryan E. Feldman, Nori Kanazawa, Robert Carroll, Yair Movshovitz-Attias, Jonathan T. Barron, Yael Pritch, and Marc Levoy. Synthetic depth-of-field with a single-camera mobile phone. *CoRR*, abs/1806.04171, 2018.

[11] Andrey Ignatov, Radu Timofte, Andrei Kulik, Seungsoo Yang, Ke Wang, Felix Baum, Max Wu, Lirong Xu, and Luc Van Gool. AI Benchmark: All about deep learning on smartphones in 2019. 2019.

[12] Get Started with TensorFlow Lite.

[13] Mark Sandler, Andrew G. Howard, Menglong Zhu, Andrey Zhmoginov, and Liang-Chieh Chen. Inverted residuals and linear bottlenecks: Mobile networks for classification, detection and segmentation. *CoRR*, abs/1801.04381, 2018.

[14] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew G. Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. *CoRR*, abs/1712.05877, 2017.

[15] TensorFlow Lite on GPU.

[16] Artistic Style Transfer with TensorFlow Lite.

[17] Converter Python API Guide.

[18] ML Kit for Firebase.

[19] ML Kit for Firebase - Custom Models.

[20] Jonathan Huang, Vivek Rathod, Chen Sun, Menglong Zhu, Anoop Korattikara, Alireza Fathi, Ian Fischer, Zbigniew Wojna, Yang Song, Sergio Guadarrama, and Kevin Murphy. Speed/accuracy trade-offs for modern convolutional object detectors. *CoRR*, abs/1611.10012, 2016.

[21] Tensorflow. github.com/tensorflow/magenta: Fast Style Transfer for Arbitrary Styles.

[22] Painter by Numbers (PBN).

[23] Describable Textures Dataset (DTD).

[24] ImageNet.

[25] Compare Pixel 4 and Pixel 4 XL Tech Specs.

[26] Samsung Galaxy S10: Smartphone with a 855.

[27] Specifications: Samsung Galaxy S10e, S10 S10.

[28] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. 2014.

[29] Introducing TF-GAN: A lightweight GAN library for TensorFlow 2.0.

[30] Mobile Operating System Market Share Worldwide.

[31] ArthurHub. github.com/arthurhub/android-image-cropper: Image Cropping Library for Android.